



Cube Camera Quick Reference

Cube Camera Quick Reference

For implementing video reflections in Freque Three.js plugins

When to Use

YES - Use Cube Cameras For: - Video reflections on metallic/chrome surfaces - Real-time environment mapping with video - Dynamic reflections that update every frame - Three.js r120+ video environment maps

NO - Don't Need Cube Cameras For: - Static background image reflections (use EquirectangularReflectionMapping) - Direct video textures on surfaces (use UVMapping + flipY) - Non-reflective materials

5-Step Implementation

Step 1: Create Cube Render Target

```
this.cubeRenderTarget = new THREE.WebGLCubeRenderTarget(2048, {  
  format: THREE.RGBAFormat,  
  generateMipmaps: true,  
  minFilter: THREE.LinearMipmapLinearFilter,  
  magFilter: THREE.LinearFilter  
});  
this.cubeRenderTarget.texture.encoding = THREE.sRGBEncoding;
```

Resolution Guide: - 4096: Ultra quality (~402 MB GPU memory) - 2048: High quality (~100 MB) - **Recommended** - 1024: Good quality (~25 MB)

Step 2: Create Cube Camera

```
this.cubeCamera = new THREE.CubeCamera(0.1, 1000, this.cubeRenderTarget);  
this.cubeCamera.position.set(0, 0, 0);  
this.cubeCamera.layers.enable(1); // See layer 1  
this.scene.add(this.cubeCamera);  
this.cubeMapTexture = this.cubeRenderTarget.texture;
```

Step 3: Create Video Sphere

```
// Large inverted sphere
const geometry = new THREE.SphereGeometry(500, 120, 80);
geometry.scale(-1, 1, 1);

// Video texture
const videoTexture = new THREE.VideoTexture(videoElement);
videoTexture.mapping = THREE.EquirectangularReflectionMapping;
videoTexture.encoding = THREE.sRGBEncoding;

// Material and mesh
const material = new THREE.MeshBasicMaterial({ map: videoTexture });
this.videoSphere = new THREE.Mesh(geometry, material);
this.videoSphere.layers.set(1); // Hidden from main camera
this.videoSphere.rotation.y = Math.PI / 2;
this.videoSphere.rotation.x = -35 * (Math.PI / 180);
this.scene.add(this.videoSphere);
```

Step 4: Apply to Materials

```
// For chrome/metallic materials
material.envMap = this.cubeMapTexture;
material.metalness = 1.0;
material.roughness = 0.0;
material.envMapIntensity = 0.5;
material.needsUpdate = true;
```

Step 5: Update Every Frame

```
onRender(deltaTime, timestamp, sharedAudioData) {  
  if (this.videoElement && !this.videoElement.paused) {  
    // Update video texture  
    this.videoSphere.material.map.needsUpdate = true;  
  
    // Hide reflective objects  
    this.spheres.forEach(s => s.mesh.visible = false);  
  
    // Update cube camera  
    this.videoSphere.visible = true;  
    this.cubeCamera.update(this.renderer, this.scene);  
  
    // Restore visibility  
    this.spheres.forEach(s => s.mesh.visible = true);  
  }  
  
  this.renderer.render(this.scene, this.camera);  
}
```

Advanced: Dynamic Positioning

```
// Calculate average position
let avgX = 0, avgY = 0, avgZ = 0;
this.spheres.forEach(s => {
    avgX += s.mesh.position.x;
    avgY += s.mesh.position.y;
    avgZ += s.mesh.position.z;
});
avgX /= this.spheres.length;
avgY /= this.spheres.length;
avgZ /= this.spheres.length;

// Move cube camera to average
this.cubeCamera.position.set(avgX, avgY, avgZ);
this.cubeCamera.update(this.renderer, this.scene);
```

Layer System

```
// Layer 0: Visible to main camera (user sees)
// Layer 1: Hidden from main camera (cube camera sees)

// Hide from main camera, show to cube camera
this.videoSphere.layers.set(1);

// Show to both cameras (toggle visibility)
this.videoSphere.layers.set(this.bgSphereVisible ? 0 : 1);
```

Performance Optimization

Quality Capture

```
// Increase quality during cube camera update
const originalPixelRatio = this.renderer.getPixelRatio();
this.renderer.setPixelRatio(Math.min(originalPixelRatio * 2.0, 2.0));
this.cubeCamera.update(this.renderer, this.scene);
this.renderer.setPixelRatio(originalPixelRatio);
```

Anisotropic Filtering

```
const maxAnisotropy = this.renderer.capabilities.getMaxAnisotropy();
this.cubeMapTexture.anisotropy = maxAnisotropy;
```

Update Frequency

```
// Update every frame (best quality)
this.cubeCamera.update(this.renderer, this.scene);

// Update every N frames (performance)
if (this.frameCount % 2 === 0) {
    this.cubeCamera.update(this.renderer, this.scene);
}
this.frameCount++;
```

Common Issues

Problem: Black spheres, no reflections

Solutions: 1. Check video element is playing: `!video.paused && video.readyState >= 2` 2. Verify video texture needsUpdate: `videoTexture.needsUpdate = true` 3. Ensure cube camera updates: `cubeCamera.update(renderer, scene)` 4. Check video sphere is visible to cube camera: `videoSphere.layers.set(1)` 5. Verify cube camera sees layer 1: `cubeCamera.layers.enable(1)`

Problem: Reflections wrong color/brightness

Solutions: 1. Set consistent encoding: `THREE.sRGBEncoding` everywhere 2. Renderer: `renderer.outputEncoding = THREE.sRGBEncoding` 3. Cube target: `cubeRenderTarget.texture.encoding = THREE.sRGBEncoding` 4. Video texture: `videoTexture.encoding = THREE.sRGBEncoding`

Problem: Poor performance

Solutions: 1. Reduce cube map resolution: Use 1024 or 2048 instead of 4096 2. Update less frequently: Every 2-3 frames instead of every frame 3. Use shared cube camera: Not per-object cameras 4. Hide objects during capture: `mesh.visible = false` 5. Only update when needed: Check video is playing and mode is reflection

Selectors

```
// Video element
const video = document.querySelector('#bgVideo') ||
    document.querySelector('#bgVideoPlaceholder') ||
    document.querySelector('video');

// Background image
const bgDiv = document.querySelector('#bgImage');
const style = window.getComputedStyle(bgDiv);
const bgImage = style.backgroundImage;
const urlMatch = bgImage.match(/url\(['"]?([^\"]+)[\"']?\)/);
const imageUrl = urlMatch[1];
```

Complete Minimal Example

```

// In onInitialize()
const video = document.querySelector('#bgVideo');

// 1. Cube render target
this.cubeRT = new THREE.WebGLCubeRenderTarget(2048, {
    format: THREE.RGBAFormat,
    generateMipmaps: true,
    minFilter: THREE.LinearMipmapLinearFilter
});
this.cubeRT.texture.encoding = THREE.sRGBEncoding;

// 2. Cube camera
this.cubeCam = new THREE.CubeCamera(0.1, 1000, this.cubeRT);
this.cubeCam.position.set(0, 0, 0);
this.cubeCam.layers.enable(1);
this.scene.add(this.cubeCam);

// 3. Video sphere
const geom = new THREE.SphereGeometry(500, 120, 80);
geom.scale(-1, 1, 1);
const tex = new THREE.VideoTexture(video);
tex.mapping = THREE.EquirectangularReflectionMapping;
tex.encoding = THREE.sRGBEncoding;
const mat = new THREE.MeshBasicMaterial({ map: tex });
this.vidSphere = new THREE.Mesh(geom, mat);
this.vidSphere.layers.set(1);
this.vidSphere.rotation.y = Math.PI / 2;
this.scene.add(this.vidSphere);

// 4. Apply to material
myMaterial.envMap = this.cubeRT.texture;
myMaterial.metalness = 1.0;
myMaterial.roughness = 0.0;

// In onRender()

```

```
if (video && !video.paused) {  
    this.vidSphere.material.map.needsUpdate = true;  
    this.spheres.forEach(s => s.mesh.visible = false);  
    this.vidSphere.visible = true;  
    this.cubeCam.update(this.renderer, this.scene);  
    this.spheres.forEach(s => s.mesh.visible = true);  
}  
this.renderer.render(this.scene, this.camera);
```

Asking Claude

Template Request

Claude, create a Three.js plugin with chrome spheres reflecting video.

Requirements:

- Use cube camera system for video reflections
- Support both video (#bgVideo) and background image (#bgImage)
- Dual mode: direct texture AND chrome reflection
- Shared cube camera with dynamic positioning (average sphere position)
- 2048 cube map resolution for balance
- Proper sRGB encoding throughout
- Layer system to hide/show video sphere
- Only update cube camera when in reflection mode and video playing

Controls:

- Toggle: Reflection vs Direct Texture
- Toggle: Video vs Background Image
- Toggle: Show/Hide Background Sphere
- Slider: Metalness 0-1, default 1.0
- Slider: Roughness 0-1, default 0.0
- Slider: Reflection Intensity 0-3, default 0.5

Reference Implementation

See: Chrome Spheres plugin (`chromospheres-freque-plugin.js`)

Key Files: - Lines 564-663: Cube camera setup - Lines 1046-1126: Cube camera update in `onRender` - Lines 734-772: Material application (dual modes)

Memory Costs

```
Cube Map Resolution → GPU Memory  
4096 × 6 faces × 4 bytes = ~402 MB  
2048 × 6 faces × 4 bytes = ~100 MB  
1024 × 6 faces × 4 bytes = ~25 MB
```

Recommendation: Start with 2048, reduce to 1024 if needed.

Render Cost

```
Shared Cube Camera:  
6 render passes per frame (one for each cube face)
```

```
Per-Object Cube Cameras (N objects):  
6N render passes per frame
```

```
Recommendation: ALWAYS use shared camera
```

Control Update Behavior

Immediate vs Deferred Updates

Controls with Immediate Effect: - Sphere Count - recreates all spheres instantly - Max Size - updates geometry immediately - Spread - repositions spheres instantly - Metalness/Roughness - updates materials immediately - Reflection Intensity - updates materials immediately

Controls with Deferred Effect: - Float Speed - affects animation speed (next frame) - Audio toggles - affects processing (next frame) - Camera controls - updates camera (next frame)

Why It Matters: Immediate updates may cause a brief visual “pop” when adjusted during playback. This is intentional for shape/position changes to give instant user feedback.

Advanced: Video Intensity Adjustment

Video cube maps are automatically reduced to 80% intensity:

```
// Automatic in updateSphereMaterials()
if (mappingMode === 'reflection' && envMapSource === 'video') {
    material.envMapIntensity = this.envMapIntensity * 0.8;
}
```

This maintains consistent appearance between video and image sources (video cube maps render brighter).

Advanced: Float Speed Scaling

Float Speed uses non-linear scaling for better control:

```
// 0-3 → 0-1.0x (slow to normal)
// 3-10 → 1.0-2.0x (normal to double)
const speedFactor = value <= 3
    ? value / 3.0
    : 1.0 + ((value - 3) / 7.0);
```

Center point at 3 = normal speed (1.0x) for easy reset.

Quick Reference v1.0 - Based on Chrome Spheres v1.3.0

Freque Plugin Development Documentation

© 2025 Freque - Professional Audio Visualization Platform